

Chapter 11

Life Cycle-Based Testing

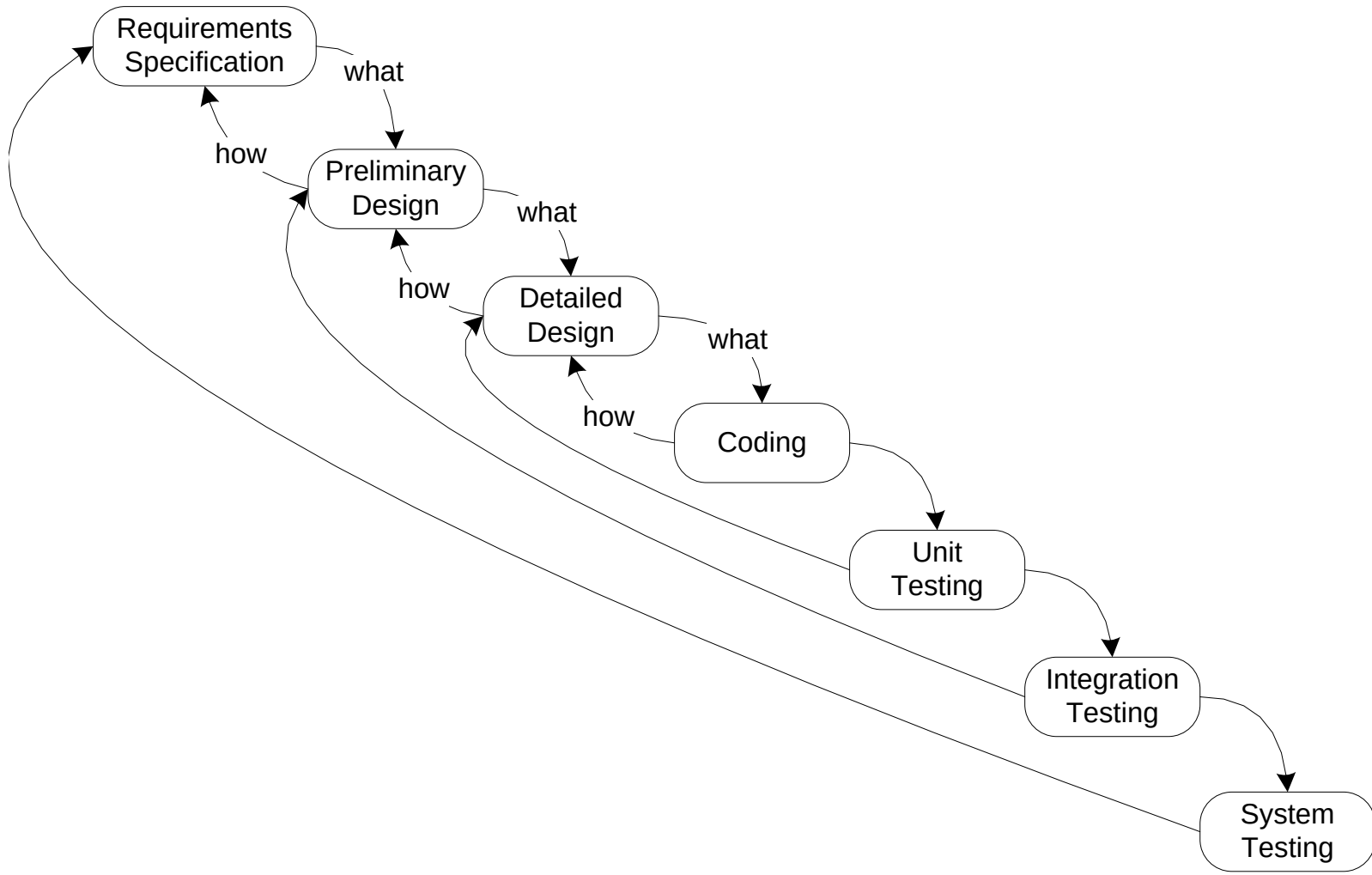


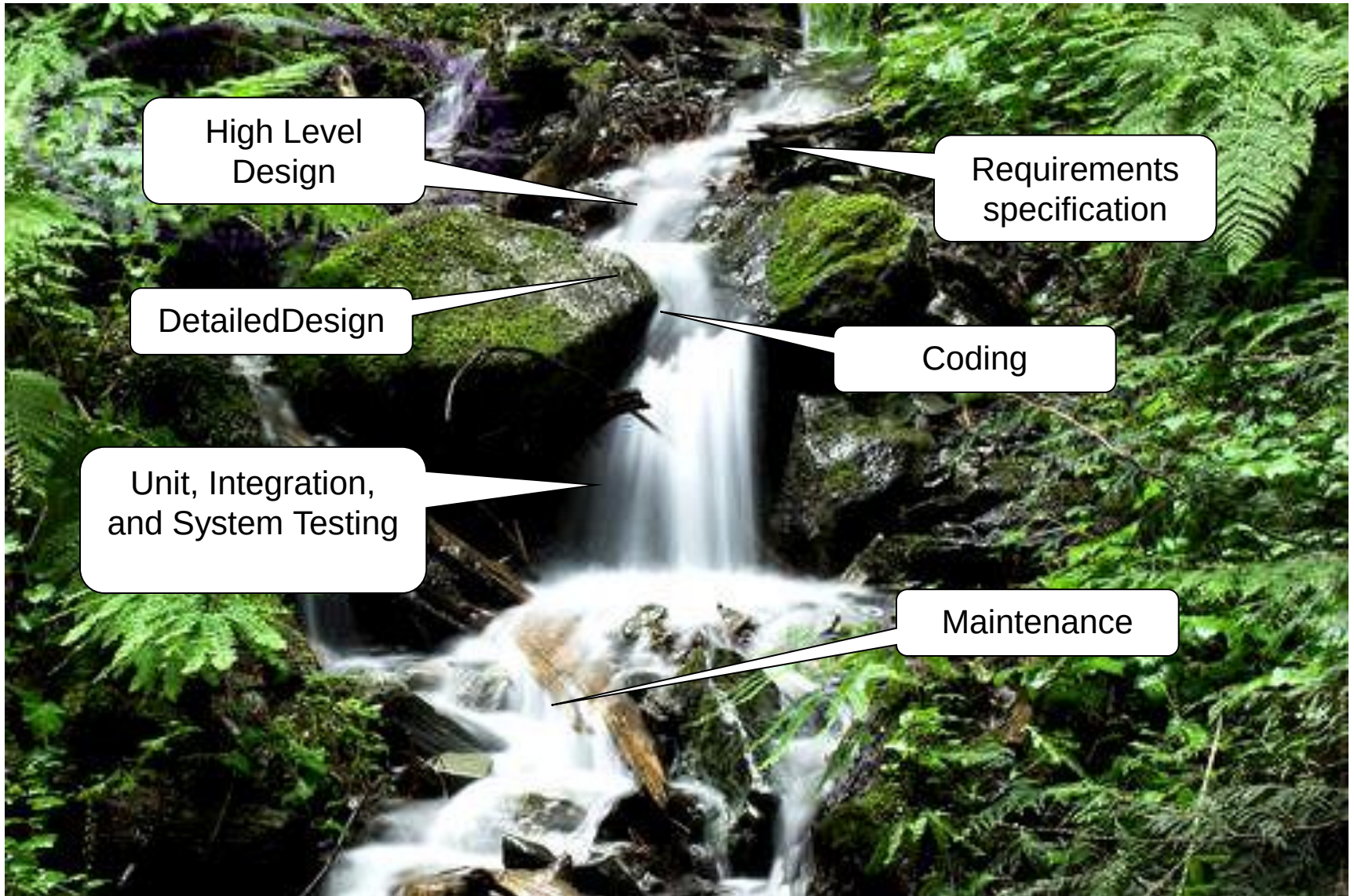
Levels and Life Cycle Models

- Levels of testing depend primarily on the software life cycle used.
- BUT, most forms of testing levels are derived from the V-Model version of the good, old Waterfall Model.
- Iterative models introduce the need for regression testing.
- System testing is greatly enhanced when an executable specification is used.



The Waterfall Lifecycle



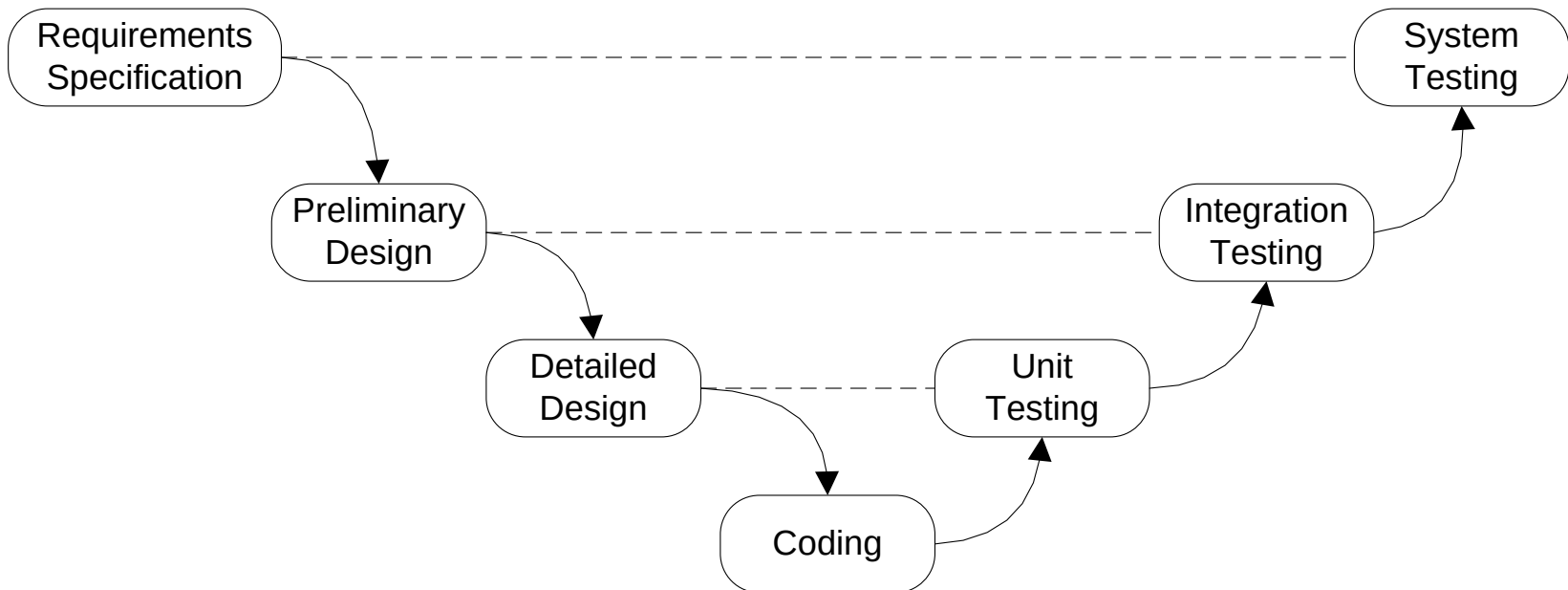


The Waterfall Lifecycle Model

- Earliest lifecycle model
 - NATO conference in 1968
 - probably “in use” before that
- Very sequential—output of one phase expresses “what” must be done in the next phase
- Strong emphasis/importance of design
- What-How cycles are natural points for software technical inspections
- Basis for unit, integration, and system levels of testing (see V-Model)
- Basis for many spinoff lifecycle models



The V-Model



Evaluation of the Waterfall Model

- Advantages

- hierarchical structure maps nicely into large projects
- phases have well-defined end products
 - (see IBM's entry and exit criteria)
- Unit level work can be done in parallel, reducing overall project interval
- Works best with procedural code

- Disadvantages

- Extremely long feedback cycle for customer
- Very late synthesis (begins at integration testing)
- Staff limitations may not support the advantage of massive parallel development at the unit level
- Requires “perfect foresight”, otherwise early faults propagate
- Awkward for o-o code

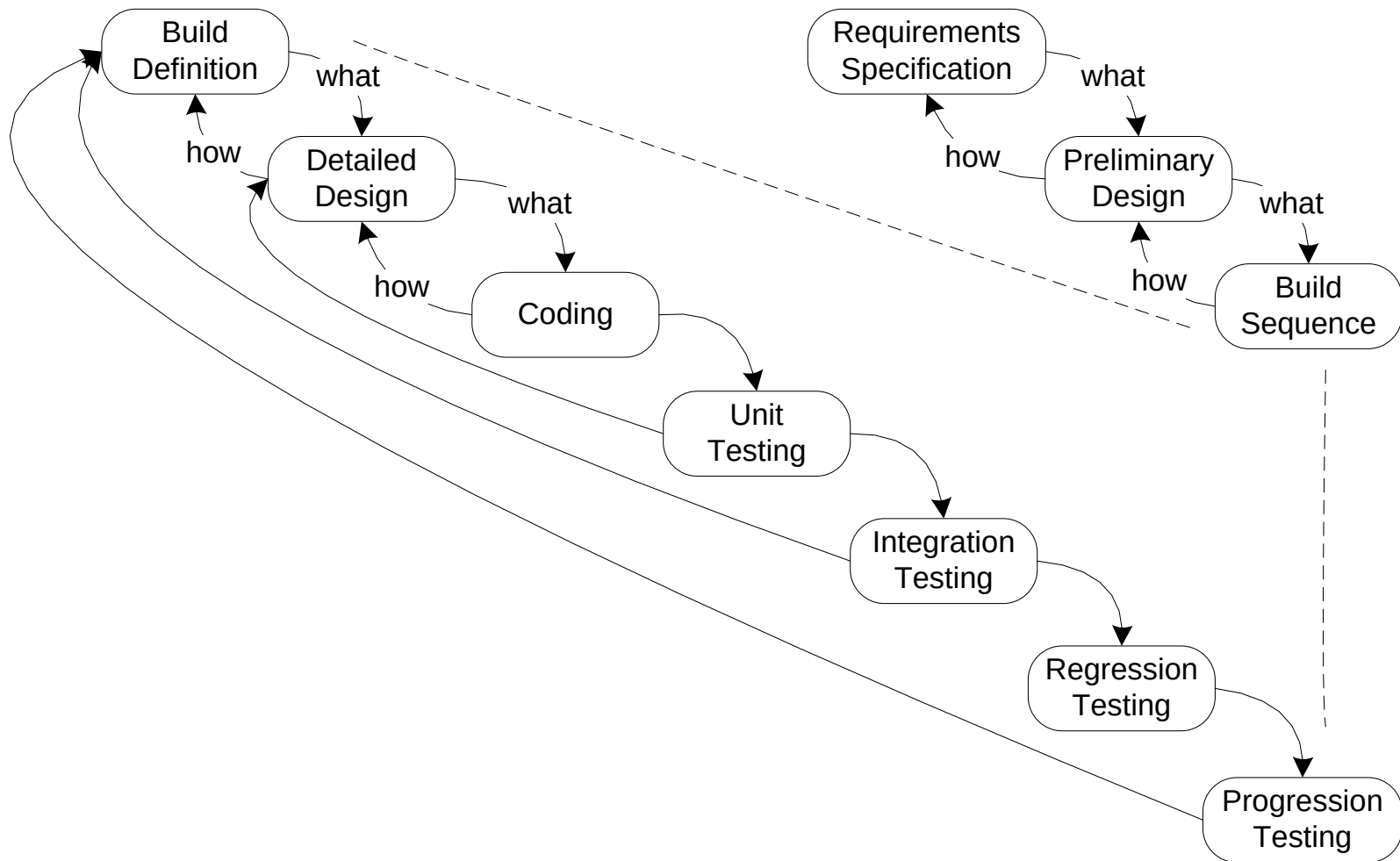


Spin-off Models

- Practitioner responses to waterfall limitations
- Iterative Development
- The Spiral Model
- Rapid Prototyping
- Executable Specification
- Agile models
 - Scrum
 - eXtreme Programming (XP)
 - Test-Driven Development
- Two promising hybrids
 - Agile Model-Driven Development (AMDD)
 - Model-Driven Agile Development (MDAD)



Iterative Development



Iterative Development

- Preserves a single high level design phase
 - amortizing design across increments is risky.
 - Early design decisions may eliminate later design choices
 - defines the sequence and content of “builds” (or increments)
- Builds create the need for regression testing
- Preserves the advantages of Waterfall, AND
- Responds to Waterfall defects
 - staffing limitations
 - late synthesis
 - long feedback cycle with customer



The Spiral Model

- Proposed by Barry Boehm in 1988
- Very similar to the Iterative Model
 - builds are selected based on risk and feasibility
- Pictured as an expanding spiral superimposed on the x-y plane (see internet for copyrighted images)
- “Quadrants” correspond to a sequence of build activities
 - determining objectives
 - risk analysis
 - development and test
 - planning next increment
- Single high level design phase is lost (which might be an inherent risk)
- Workable for o-o projects

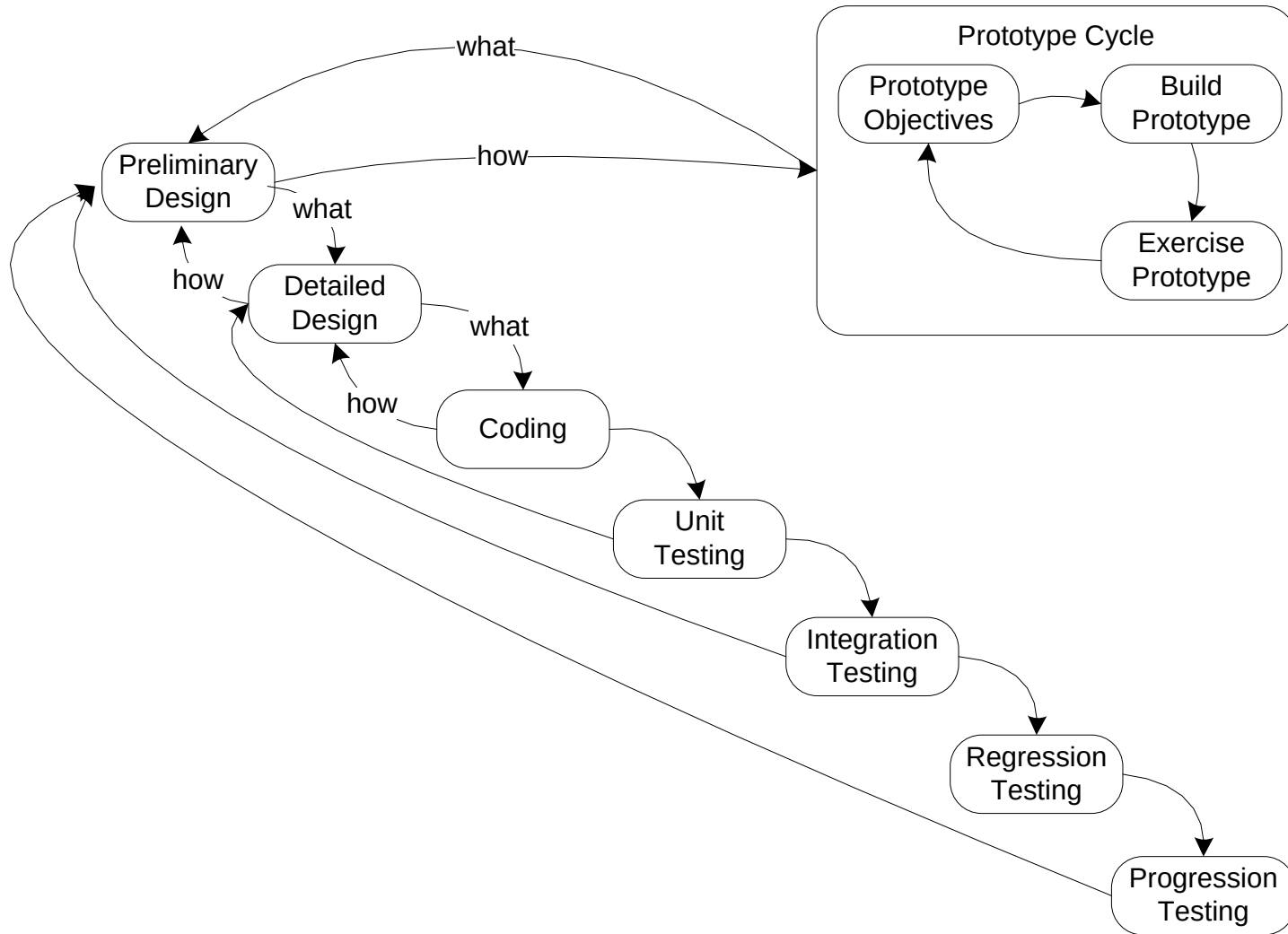


“Perfect Foresight?”

- Waterfall and the iterative variations have no answer for the customer who does not have a clear, complete idea of what is needed.
- “Requirements Elicitation” is the process of helping customers and developers reach a common understanding of a proposed system.
- Three lifecycle responses...
 - Rapid Prototyping
 - Executable Specification
 - the Agile methods



Rapid Prototyping

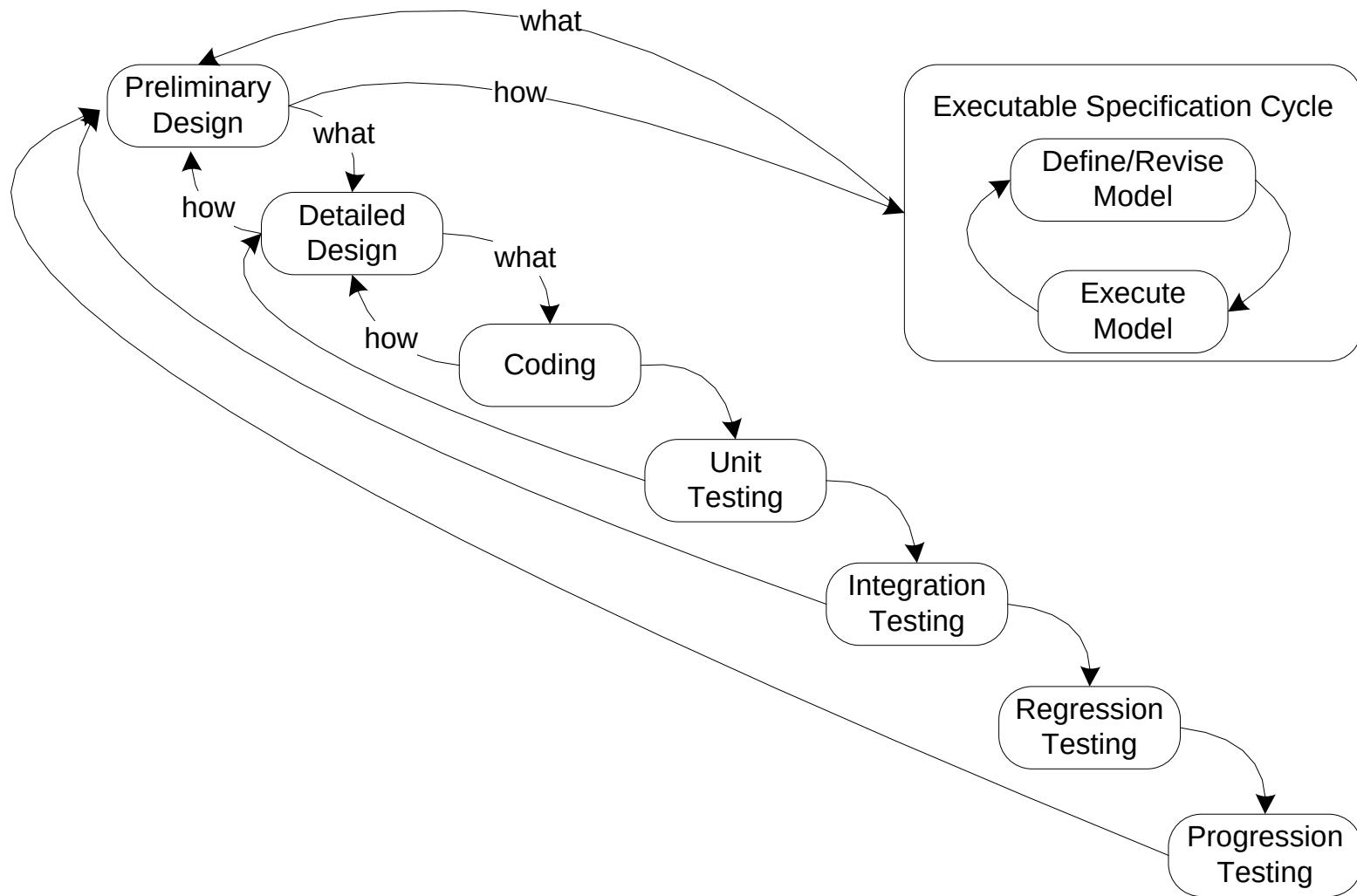


Rapid Prototyping

- Helps customer identify needs and defects
 - “I’ll know what I want when I see it.”
 - provides the “does view” that customers appreciate
 - ideal to give the “look and feel” of menu-driven systems
 - modify prototype per customer feedback
 - Sometimes done for feasibility
- Advantages:
 - improved and early feedback with customer
 - better basis for design
- Keep or dispose?
 - once it has served its purpose, the prototype can be archived.
 - possible to use to identify test scenarios



Executable Specification



Executable Specification

- Very similar to Rapid Prototyping
 - early feedback
 - “look and feel”
- Best for event-driven systems
- Executable model is the specification
 - finite state machine
 - StateChart
 - some form of Petri Net
- Need an “engine”
 - model is executed by the engine
 - usually interactively with customer

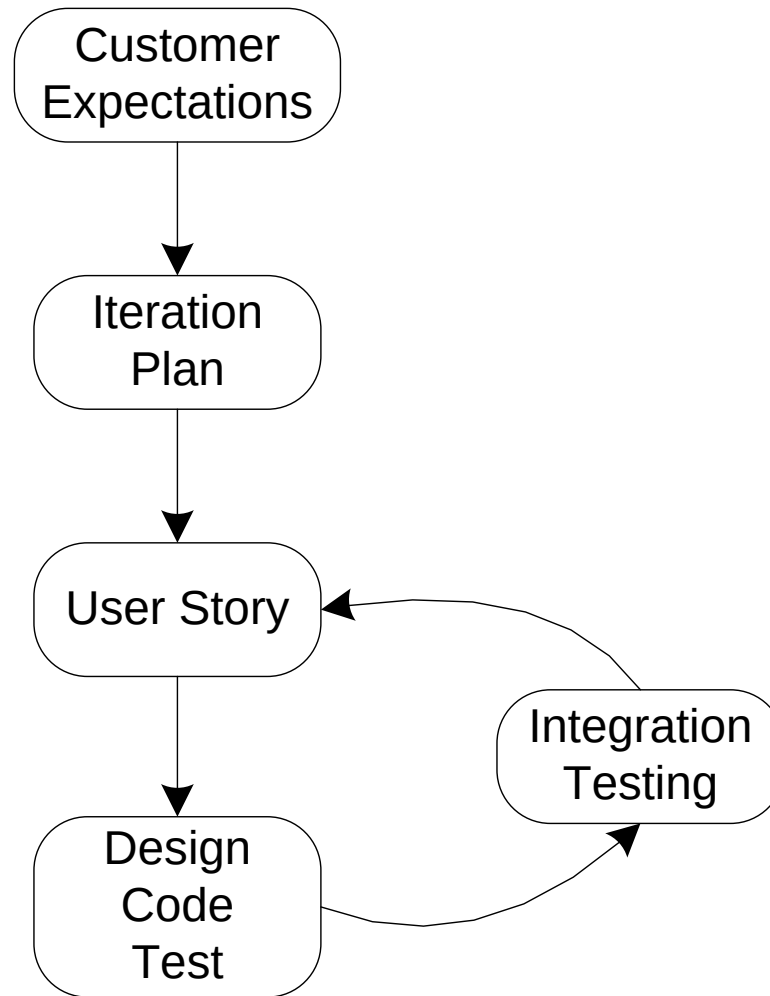


Evaluation of Executable Specification

- Intended for “reactive systems” (event-driven)
- Advantages
 - early feedback
 - automatic generation of system test cases
 - can be used for operator training
 - support for early analysis
- Disadvantages
 - modeling can be difficult
 - training may be necessary
 - engine can be expensive



Generic Agile Lifecycle

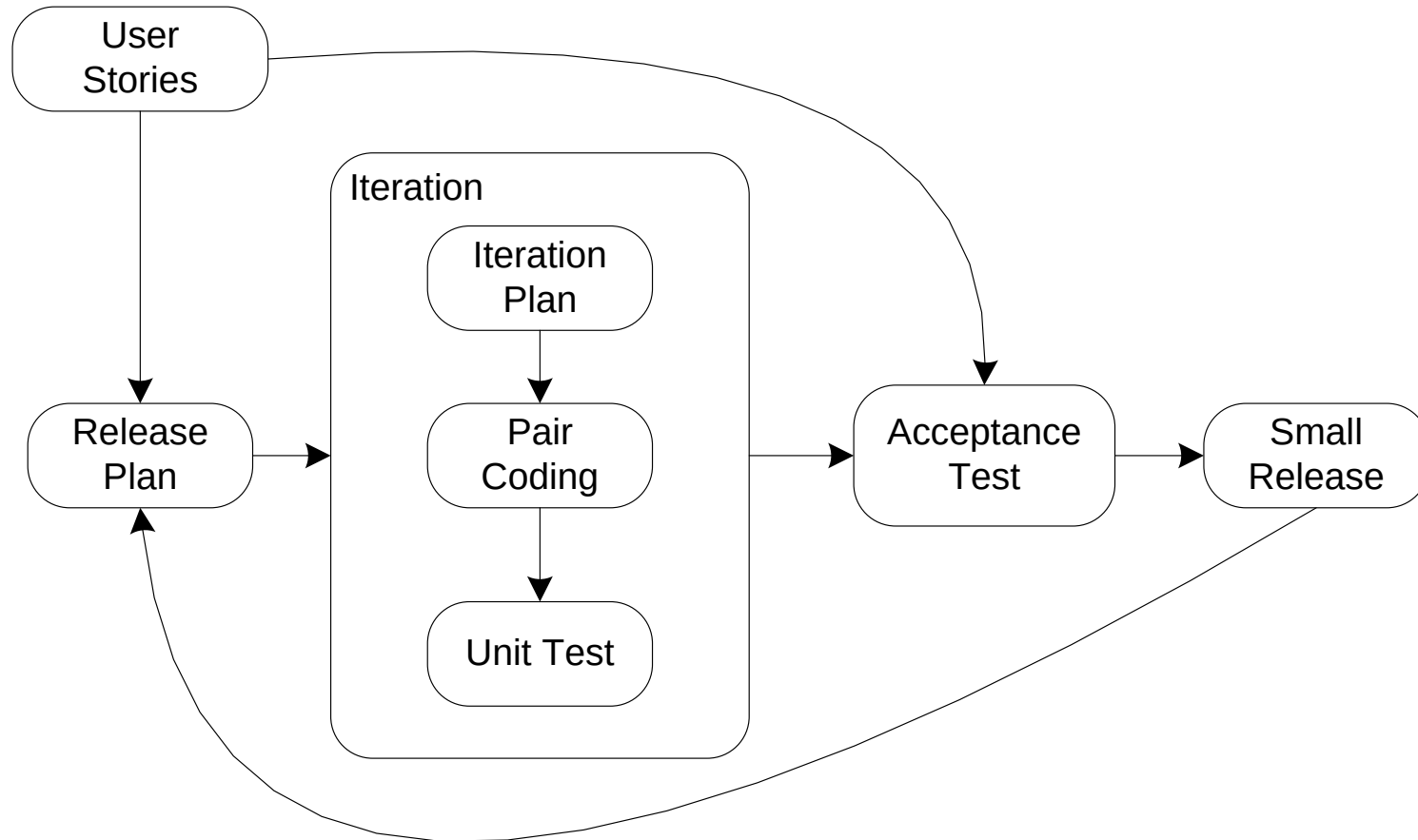


Agile Development

- Best response to the customer who does not know what is needed, *i.e.*, “perfect foresight”
- (see the Agile Manifesto)
- Customer-driven, hence excellent customer feedback
- Short increments (early synthesis)
- We look at
 - eXtreme Programming (XP)
 - Test-Driven Development (TDD)
 - Behavior-Driven Development (BDD)
 - Use Case Based Development
 - Scrum



eXtreme Programming

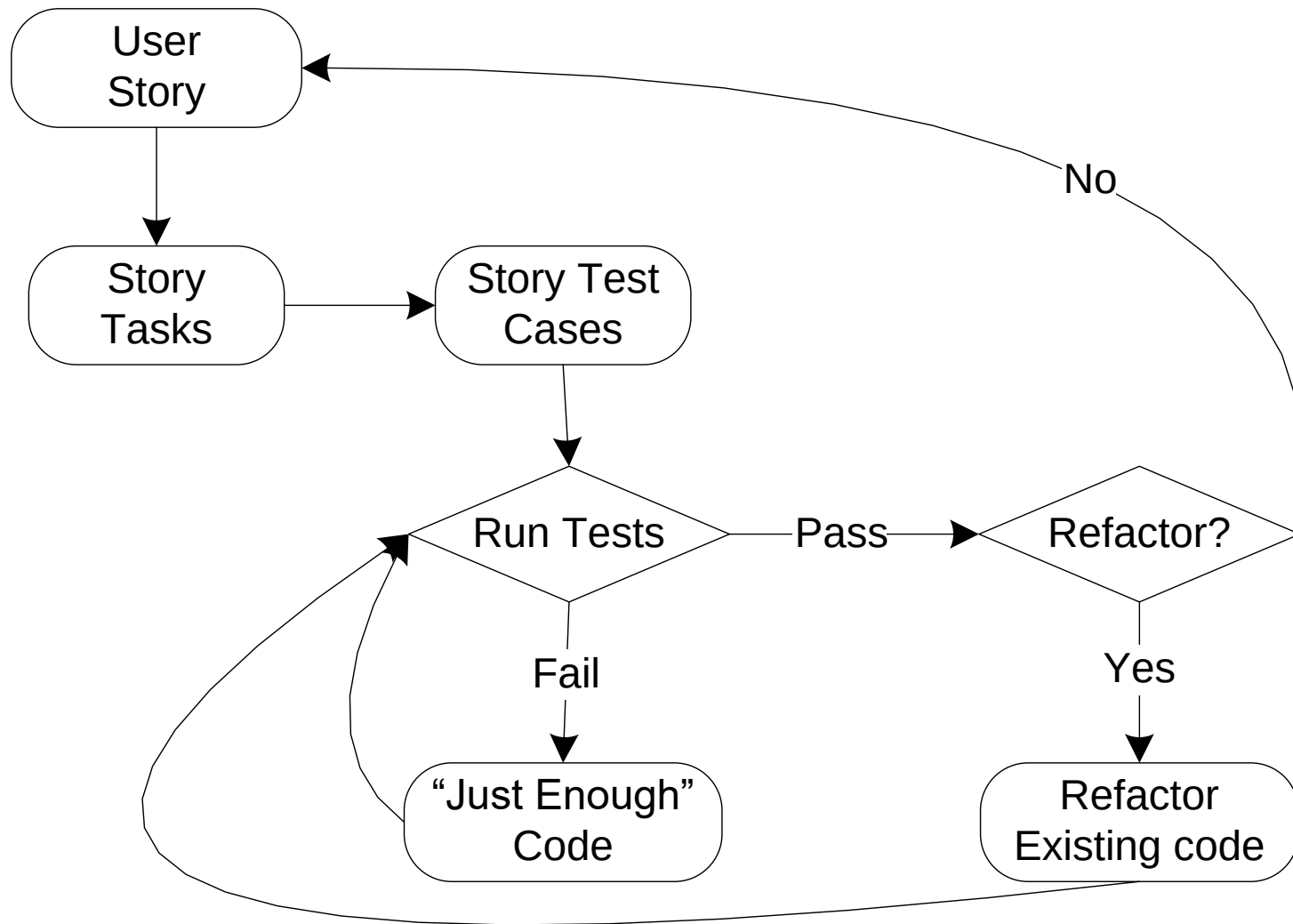


eXtreme Programming

- Kent Beck, 1996
- Distinguishing characteristic: pair programming
 - one person has the detailed view (and the keyboard)
 - partner has the overall view, and acts as a constant reviewer
 - roles can change
- Bottom-up development precludes a single, high level design phase
 - (but that might not be possible with an uncertain customer anyway)
 - Works well for o-o projects



Test-Driven Development (TDD)



Test-Driven Development (TDD)

- Extreme case of agile development
- Sometimes called “Test First Development”
- Bottom-up development based on test cases
 - derived from customer-provided user stories
 - very quick feedback
- Very small increments
 - early synthesis
 - excellent fault isolation
 - refactoring results in clean code
- BUT, no opportunity for a comprehensive design



Example: a Boolean Function to Determine Leap Years

- Definition: A year is a **leap year** if it is a multiple of 4, but century years are leap years only if they are multiples of 400.
- We compare Model-Driven and Test-Driven approaches.

Rules	1	2	3	4	5	6	7	8
c1. year is a multiple of 4	T	T	T	T	F	F	F	F
c2. year is a century year	T	T	F	F	T	T	F	F
c3. year is a multiple of 400	T	F	T	F	T	F	T	F
a1. logically impossible			X		X	X	X	
a2. year is a common year		X						X
a3. year is a leap year	X			X				
test case: year =	2000	1900		2008				2011



Sequence of TDD User Stories

- User Story 1: A year divisible by 4 is a leap year
 - Test Case: input = 2020 Expected output = true
- User Story 2: A year not divisible by 4 is a common year
 - User story 1 plus
 - Test Case: input = 2021 Expected output = false
- User Story 3: A century year not divisible by 4 is a common year
 - User story 2 plus
 - Test Case: input = 1900 Expected output = false
- User Story 4: A century year divisible by 400 is a leap year
 - User story 3 plus
 - Test Case: input = 2000 Expected output = true



Java Code for User Story 1

```
public static boolean isLeap(int year) {  
    // Task 1 No code  
    //A year divisible by 4 is a leap year  
    //Test Case: input = 2020 Expected output = True  
}
```

```
public static boolean isLeap(int year) {  
    boolean c1  
    // Task 1 New code  
    //A year divisible by 4 is a leap year  
    //Test Case: input = 2020 Expected output = true  
    c1 = (year % 4 == 0);        // leap years are divisible by 4  
    If c1 {  
        return true;    //rule 1  
    }  
}
```

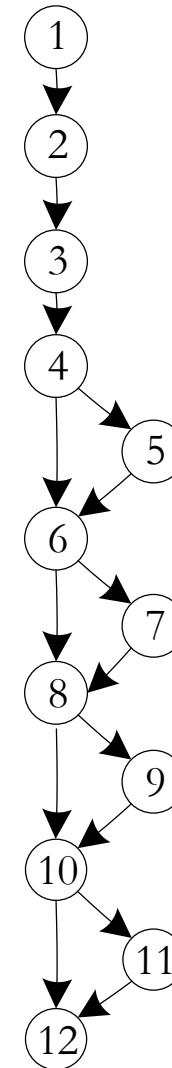
Exercise: Write Code for User Stories 2 and 3



Java Code And Program Graph for User Story 4

```
public static boolean isLeap(int year) {  
    boolean c1; boolean c2; boolean c3;
```

```
1.  c1 = (year % 4 == 0);  
    // leap years are divisible by 4  
2.  c2 = (year % 100 == 0);  
    // but century years are common years  
3.  c3 = (year % 400 == 0);  
    // unless they are divisible by 400  
  
4.  if (c1 && c2 && c3) {  
5.    return true; // rule 1  
6.  } else if (c1 && c2 && !c3) {  
7.    return false; // rule 2  
8.  } else if (c1 && !c2 && !c3) {  
9.    return true; // rule 4  
10. } else  
11.   return false; // rule 8  
12. }
```



Behavior-Driven Development (BDD)

- An extension of Test-Driven Development
- Adds structure to a User Story
 - A scenario name
 - A series of conditions
 - A series of actions
- Easily transformed into a decision table.
- We begin with a refinement, the definition of a “well-formed” BDD scenario
- Illustrated with an example from Dan North (early proponent of BDD)



BDD Example and Definition

- Dan North example
 - c1: Given the account is in credit
 - c2: And the card is valid
 - c3: And the dispenser contains cash
 - c4: When the customer requests cash
 - a1: Then ensure the account is debited
 - a2: And ensure cash is dispensed
 - a3: And ensure the card is returned



Well-Formed BDD Definition

- A *well-formed BDD scenario* has the following sections and structure
 - IF (<pre-condition(s)>)
 - AND (<data condition(s)>)
 - AND (<input event sequence>)
 - THEN (action sequence)
 - AND (output event sequence)
 - AND (<post-condition(s)>)
- Note the similarity to Expanded Essential Use Cases
- And the easy conversion to a decision table



Dan North Scenario as a Decision Table

Rule	1
c1. account is in credit	T
c2. card is valid?	T
c3. dispenser contains cash	T
c4. customer requests cash	T
a1. dispense cash	x
a2. debit customer account	x
a3. return card	x



Decision Table Analysis

- Chapter 11 contains a detailed decision table analysis of the Dan North BDD scenario
- Expansion to a complete limited entry decision table with
 - 4 conditions
 - 4 actions
 - 16 rules
- After detailed analysis, including combining rules, eliminating a condition, and adding an action, we have Table 11.10 (next slide)



Decision Table 11.10

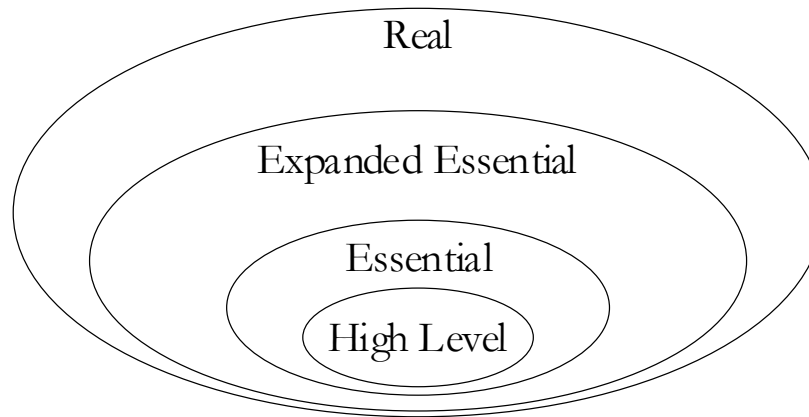
Rules	1 & 5	2 & 6	3,4,7,8	9 to 16
c2. card is valid?	T	T	T	F
c3. dispenser contains cash	T	T	F	—
c4. customer requests cash	T	F	—	—
a1. dispense cash	x	—	—	—
a2. debit customer account	x	—	—	—
a3. return card	x	x	x	x
a4. do nothing	—	x	—	x
a5: notify customer, no cash	—		x	—

- We have 4 rules, corresponding to
 - 4 BDD scenarios
 - 4 test cases
- Note how decision table analysis extends and supports BDD scenarios



Use Case Based Development

- Central to Unified Modeling Language (UML)
 - Easily understood by customers and developers
 - Capture the “Does View” of a system
 - Lead easily to test cases
- Hierarchy of use cases
 - High level (very similar to an agile User Story)
 - Essential
 - Expanded Essential
 - Real



High Level Use Case

- Content:
 - Use Case ID
 - Use Case Name
 - Description (User Story)

Essential Use Case

- Content:
 - High Level Use Case Plus
 - Interleaved sequence of input events and output actions
 - (almost equivalent to a finite state machine)



Expanded Essential Use Case

- Content:
 - Essential Use Case Plus
 - Pre-conditions
 - Post conditions
- Good Practice: develop essential use cases such they can be “connected” by their pre- and post-conditions

Real Use Case

- Content:
 - Actual values replace abstract names in an essential use case
 - Directly useful as a test case
 - Need to add actual outputs to the expected outputs



Scrum—New Terms for Existing Ideas

- (Not an acronym)
- Created in 1993 by Jeff Sutherland
- Formalized in 1995 by Ken Schwaber
- Very popular today, both in US and Europe
- Named for the importance of teamwork in rugby
- *n.b.:* material in the series of slides on Scrum is taken from
http://www.scrumalliance.org/learn_about_scrum)





Scrum—New Terms for Existing Ideas

- Three roles
 - Product owner
 - Scrum Master
 - Self-organizing team
- Three ceremonies
 - Sprint planning meeting
 - Daily scrum meeting
 - Sprint review meeting
- Three artifacts
 - Product backlog,
 - Sprint backlog
 - Burndown chart



Product Owner Responsibilities

- Define the features of the product;
 - Decide on release date and content;
 - Be responsible for the profitability of the product (ROI);
 - Prioritize features according to market value;
 - Adjust features and priority every 30 days, as needed; and
 - Accept or reject work results.
-
- Question: Product Owner = Customer?



Scrum Master Responsibilities

- Ensure that the team is fully functional and productive
- Enable close cooperation across all roles and functions
- Remove barriers
- Shield the team from external interferences
- Ensure that the process is followed (sprint planning, daily meeting, sprint review)
- Question: Scrum Master = Supervisor?



Scrum Team Responsibilities

- Cross-functional with 7 +/- 2 members
- Selects the Sprint goal and specifies work results
- Has the right to do everything within the boundaries of the project guidelines to reach the Sprint goal
- Organizes itself and its work
- Demonstrates work results to Product Owner.
- Question: Scrum Team = Development Team?



Comparison of Model Driven Development (MDD) and Test Driven Development (TDD)

- First American's view of Eagles and Mice
 - Eagles have the “big picture”
 - Mice focus on the details
 - (both views are important!)
- MDD is a rigorous, top-down approach.
- TDD is a bottom-up approach.
- (see earlier discussion of TDD and MDD)



Open Questions

- Does Agile (bottom up) Development actually result in better designs?
- Can any form of Agile Testing reveal “deep faults”?
 - e.g.: faults revealed only by data flow (define/use) testing
 - computational faults
 - time-dependent faults
- What about maintenance? Agile code has
 - well-named variables and components
 - (hopefully) been refactored carefully
 - no comments
 - test cases are the specification



A Compromise?

- How might we capture/combine the advantages of various lifecycles to avoid the known deficiencies?
- Lessons from Georg (a friend)
 - German Ph.D. mathematician and Go player
- A successful Go player must have both good strategy and good tactics
- Georg's contention...
 - Design = strategy
 - TDD = tactics

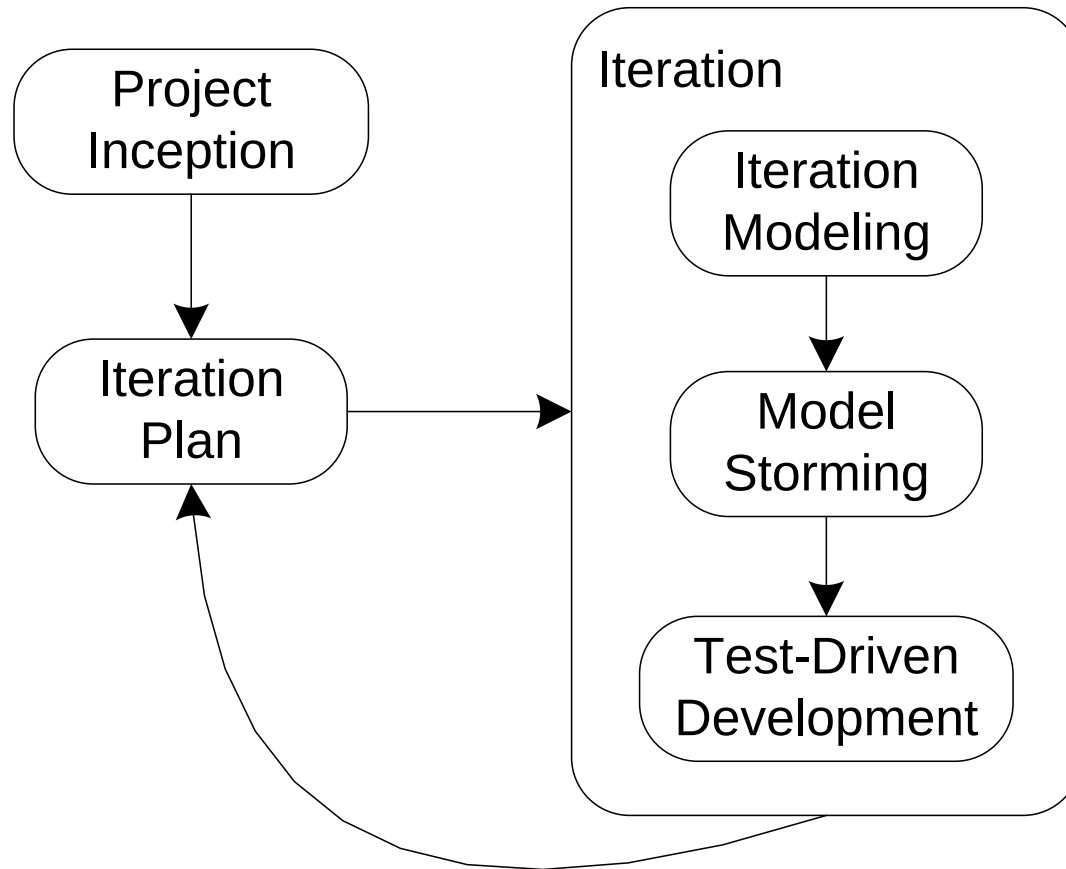


Agile Model-Driven Development

- Scott Ambler
- Model just enough for the present user story
- Design is necessary!
- BUT the modeling is still not in one phase (possibly unavoidable with the uncertain customer)



Agile Model-Driven Development



Model-Driven Agile Development

- Re-arrangement of AMDD
- Early emphasis on design as one step (thanks, Georg)
- Implementation uses Test-Driven Development



Model-Driven Agile Development

